{EPITECH}

E-TODO_

< BUILD AN API TO CREATE A TODO APP />

E-TODO

To easily handle all the tasks you have to accomplish, let's build a to-do web application.

Within this project, you're going to develop:

- ✓ a database, using *MySQL* ;
- ✓ a back-end web server, using *Node.js* ;
- ✓ a front-end, using *whatever you fancy*.

A bootstrap is here to help you! A lot!

Database

You have to manage various users and their respective tasks (todos). To ensure data persistency, reduce data redundancy and ease it management, you'll implement a database.

You must:

- ✓ name your database `etodo` ;
- ✓ create (at least) 2 tables, named `user` and `todo` ;
- ✓ write the entire db's scheme in a file named `e-todo.sql` ;
- ✓ place this last file at the root of your repository.

> Some external tools can help you build your database, then export it as a `.sql` file.

Below, you'll find specifications for your database tables and fields.
You must (always) comply with the given requirements and respect the provided constraints.

> I think frugality drives innovation, just like other constraints do.
> One of the only ways to get out of a tight box is to invent your way out.
>
> **— Jeff BEZOS —**

Your database must contain at least the following tables and fields:

- ✓ `user`:
  - `id` (mandatory, not null, auto-increments)
  - `email` (mandatory, not null, unique)
  - `password` (mandatory, not null)
  - `name` (mandatory, not null)
  - `firstname` (mandatory, not null)
  - `created_at` (current datetime by default)
- ✓ `todo`:
  - `id` (mandatory, not null, auto-increments)
  - `title` (mandatory, not null)
  - `description` (mandatory, not null)
  - `created_at` (current datetime by default)
  - `due_time` (mandatory, not null, datetime)
  - `status` (*not started* by default | *todo* | *in progress* | *done*)
  - `user_id` (mandatory)

Choose the type of each field carefully.
Think about the last `todo` field: why do you need it? Do you know what foreign keys are?

Once your scheme is created, import your file into your MySQL server.

Terminal

```
$> cat e-todo.sql | mysql -u root -p
```

Web server

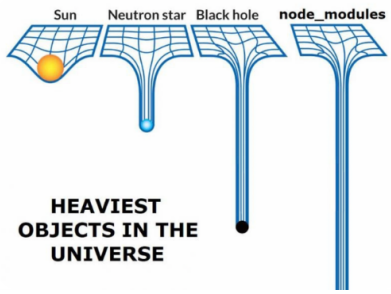
Unlike other programming languages, *Node* and *Express* let you build things your own way, meaning that you can do the same thing in tons of different ways.

This also means that your file structure can get quite messy. So, **keep your code and your architecture as clean as possible** if you don't want to get lost.

Your repository's structure should looks like this (you can adapt it to your needs):

```
|-- .env
|-- docker-compose.yml
|-- e-todo.sql
|-- backend/
|   |-- Dockerfile
|   |-- package.json    # app package file, filled with dependencies + have a starting
    script
|   `-- src/            # your source code folder for the API
|       |-- config/     # contains the files that handle the connection to the database
|       |   `-- db.js
|       |-- index.js    # main file that starts everything, calls and runs the app
|       |-- middleware/ # contains the middlewares created
|       |   |-- auth.js
|       |   `-- notFound.js
|       `-- routes/     # contains subfolders with all the needed routes
|           |-- auth/
|           |   `-- auth.js
|           |-- todos/
|           |   |-- todos.js
|           |   `-- todos.query.js
|           `-- user/
|               |-- user.js
|               `-- user.query.js
`-- frontend/
    `-- whatever is necessary here for your frontend app
```

You should have a **node_modules** folder at the root of your project.
It is not listed above, because we **NEVER PUSH** this folder.
Have you already hear about the .gitignore file?



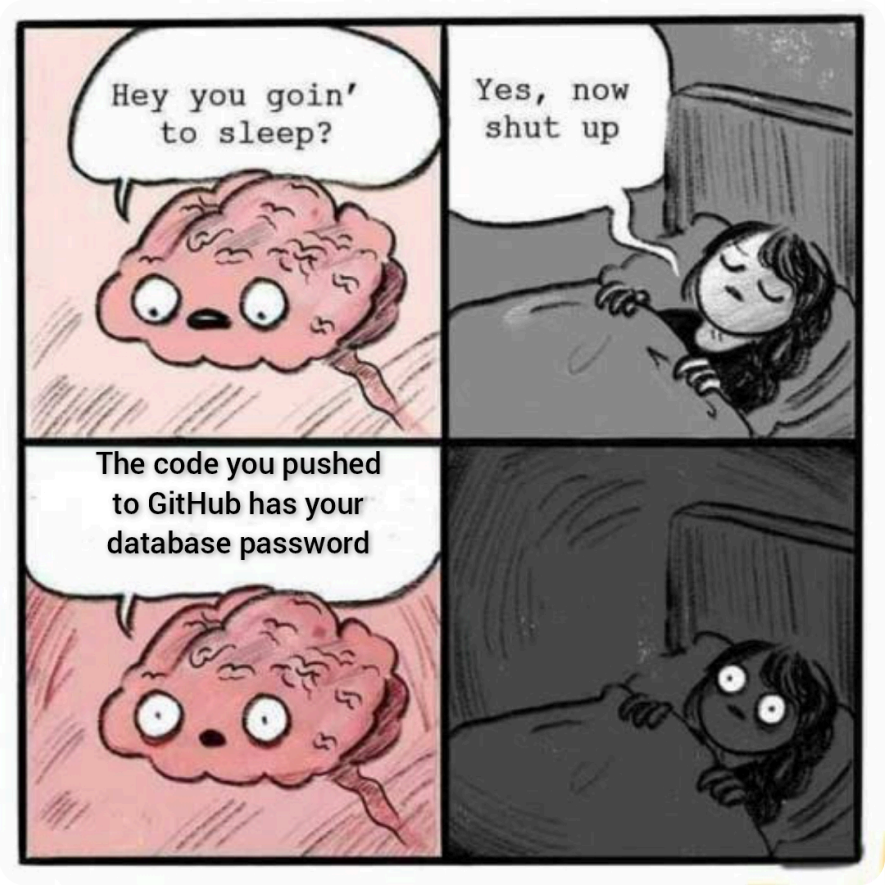
Environment variables

The `.env` file must contain all the configuration variables necessary for your project.

The required ones are:

- ✓ MYSQL_DATABASE ;
- ✓ MYSQL_HOST ;
- ✓ MYSQL_USER ;
- ✓ MYSQL_ROOT_PASSWORD ;
- ✓ PORT (used for the Express server) ;
- ✓ SECRET (used for the JSON Web Token).

NEVER push some credentials!



API

You have to use a REST API to create your CRUD system, and a JSON format to transmit the data.

We strongly advise you to read mre about APIs.

Many tools may help you building your app. Some are good, some are bad, and some are evil.

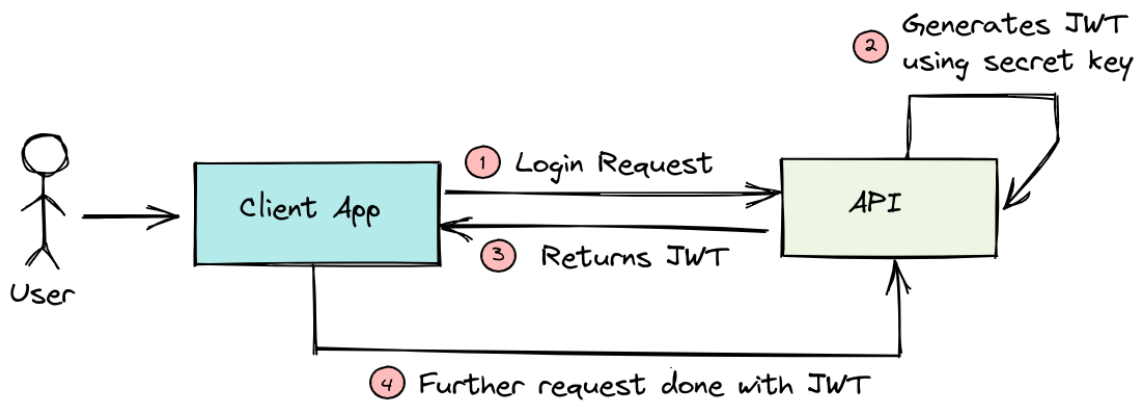
For this project, you are only authorized to use the packages listed below.

- ✓ Express ;
- ✓ mysql2 ;
- ✓ dotenv ;
- ✓ jsonwebtoken ;
- ✓ bcryptjs ;
- ✓ body-parser *(this one is optional, find out why)*.

Any other package is forbidden unless it has been validated by the pedagogical team.

Routes

We expect you to protect your routes and only make them accessibles to logged-in users.
Protected routes should receive a valid **J**SON **W**eb **T**oken in the authorization header.



Have a look at the jsonwebtoken npm package.

Below are listed all the routes expected for this project.

route	method	protected	description
/register	POST	no	register a new user
/login	POST	no	connect a user
/user	GET	yes	view all user information
/user/todos	GET	yes	view all user tasks
/users/:id or :email	GET	yes	view user information
/users/:id	PUT	yes	update user information
/users/:id	DELETE	yes	delete user
/todos	GET	yes	view all the todos
/todos/:id	GET	yes	view the todo
/todos	POST	yes	create a todo
/todos/:id	PUT	yes	update a todo
/todos/:id	DELETE	yes	delete a todo

Routes can have multiple methods, parameters or responses.
The expectations for each route are provided below.
Read the following subsections carefully.

Defaults

We suggest building a custom error-handling middleware to send the common errors directly, instead of handling it in every route. By default, each error must be processed as follows (additional details are available in the relevant section):

- ✓ If the user is not logged in: `{ "msg": "No token, authorization denied"}`
- ✓ If the user sends an invalid Token: `{ "msg": "Token is not valid"}`
- ✓ If the task or user does not exist: `{ "msg": "Not found"}`
- ✓ If user give bad parameters: `{ "msg": "Bad parameter"}`
- ✓ If there is another error: `{ "msg": "Internal server error"}`

Authentication

POST `/register` Request body:

```
{
  "email": "nao.marvin@epitech.eu",
  "name": "Marvin",
  "firstname": "Nao",
  "password": "3paulbec"
}
```

Response: `{ "token": "Token of the newly registered user"}`
Response, if the account already exists: `{ "msg": "Account already exists"}`

> ⚠ The password should not be stored as plain text! Find out the best way to hash it.

POST `/login` Request body:

```
{
  "email": "username",
  "password": "password"
}
```

Response body: `{ "token": "Token of the newly logged in user"}`
Response body, if the credentials are incorrect: `{ "msg": "Invalid Credentials"}`

Users

GET /user

Response body:

```
{
  "id": 1,
  "email": "email@test.eu",
  "password": "hashed password",
  "created_at": "2021-03-03 19:24:00",
  "firstname": "test",
  "name": "test"
}
```

GET /user/todos

Response body:

```
[
  {
    "id": 1,
    "title": "title",
    "description": "desc",
    "created_at": "2021-03-03 19:24:00",
    "due_time": "2021-03-04 19:24:00",
    "user_id": 3,
    "status": "done"
  },
  {
    "id": 2,
    "title": "title",
    "description": "desc",
    "created_at": "2021-03-05 19:24:00",
    "due_time": "2021-03-06 19:24:00",
    "user_id": 3,
    "status": "in progress"
  }
]
```

GET /users/:id and /users/:email

Response body:

```
{
  "id": 1,
  "email": "email@test.eu",
  "password": "hashed password",
```

```
  "created_at": "2021-03-03 19:24:00",
  "firstname": "test",
  "name": "test"
}
```

PUT /users/:id

Request body:

```
{
  "email": "updated_email@test.eu",
  "password": "updated_password",
  "firstname": "updated_test",
  "name": "updated_test"
}
```

Response body:

```
{
  "id": 1,
  "email": "updatedemail@test.eu",
  "password": "hashed password",
  "created_at": "2021-03-03 19:24:00",
  "firstname": "updated_test",
  "name": "updated_test"
}
```

DELETE /users/:id

Response body: `{"msg": "Successfully deleted record number: $\${id}"}`

Todos

GET /todos

Response body:

```
[
  {
    "id": 1,
    "title": "title",
    "description": "desc",
    "created_at": "2021-03-03 19:24:00",
    "due_time": "2021-03-04 19:24:00",
    "user_id": 1,
    "status": "done"
  },
  {
    "id": 2,
    "title": "title",
    "description": "desc",
    "created_at": "2021-03-05 19:24:00",
    "due_time": "2021-03-06 19:24:00",
    "user_id": 2,
    "status": "in progress"
  }
]
```

GET /todos/:id

Response body:

```
{
  "id": 2,
  "title": "title",
  "description": "desc",
  "created_at": "2021-03-05 19:24:00",
  "due_time": "2021-03-06 19:24:00",
  "user_id": 3,
  "status": "in progress"
}
```

POST /todos

Request body:

```
{
  "title": "title",
  "description": "desc",
```

```
  "due_time": "2021-03-06 19:24:00",
  "user_id": 3,
  "status": "todo"
}
```

Response body: the created todo (just like the GET todo route).

PUT `/todos/:id`

Request body:

```
{
  "title": "Updated title",
  "description": "Updated desc",
  "due_time": "2021-03-07 19:24:00",
  "user_id": 1,
  "status": "in progress"
}
```

Response body: the updated todo (just like the GET todo route).

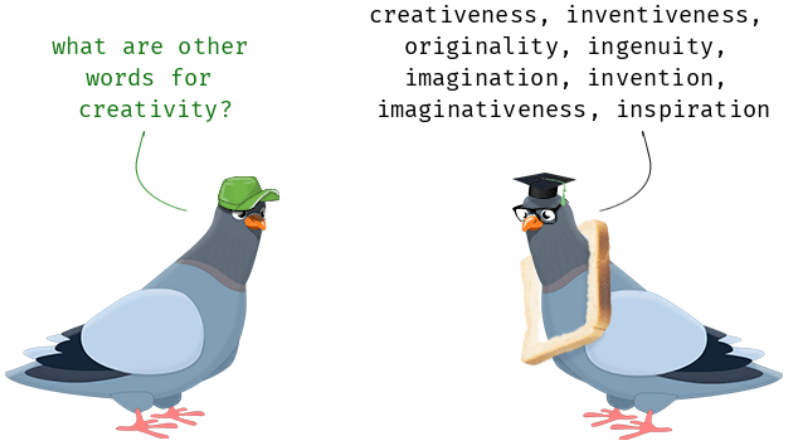
DELETE `/todos/:id`

```
{ "msg": "Successfully deleted record number: ${id}" }
```

All responses above must use an appropriate HTTP status.

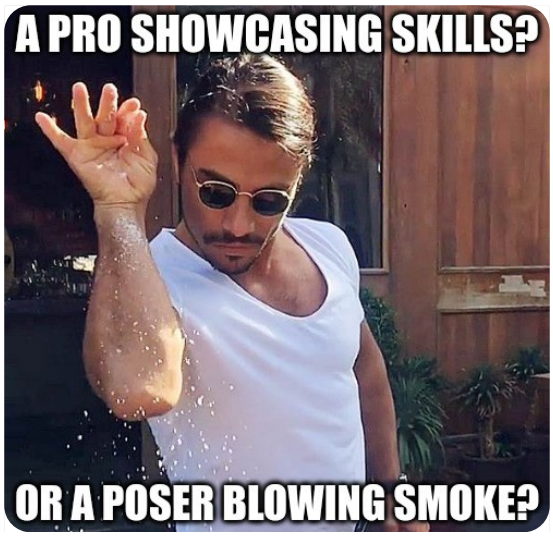
Front-end

You have to build a nice, functional front-end with an astonishing interface providing the most advanced user experience (without which your application would be unusable).
You need to **showcase your creativity**.



You have considerably more freedom for this section than others, and you're free to use any tool, such as frameworks, CSS preprocessors, frontend libraries, etc.

Beware, however: taking any action, in any form, could be detrimental.
You do NOT want to be a poser, a hack, or a clown who's out of their depth.



Material is a design system created by Google to help teams build high-quality digital experiences for Android, iOS, Flutter, and the web.
Responsively lets you preview your design on multiple devices and screen sizes.

Because The UX of your application is just as important as its functionalities, many design and frontend aspects must be seriously considered and thought through, such as:

- ✓ creation of a visual identity ;
- ✓ navigation ergonomics ;
- ✓ readability ;
- ✓ intuitiveness ;
- ✓ responsiveness ;
- ✓ etc.

This reflection is an integral part of your application's design process.

Your application's visual identity and user interface will be evaluated, just like the quality of the user experience. As such, we will check if you respect the chosen platform's guidelines such as:

- ✓ the choice and coherence of style (colors, icons, typography, etc.) ;
- ✓ the usage of space (size, placement, density, etc.) ;
- ✓ the choice of elements (buttons, lists, menus, etc.) ;
- ✓ your project's identity and the cohesion of it all.

You can consider some/all (?) of the ideas below for your interface:

- ✓ allow users to sort their tasks by drag-and-drop ;
- ✓ allow users to choose a theme (dark mode, color-blind, etc.) ;
- ✓ offer several visual presentations of tasks (board, table, timeline, kanban, etc.) ;
- ✓ implement a responsive design to ensure a good experience on mobile and desktop ;
- ✓ allow users to customize the app, by proposing several CSS styles, colors, fonts and sizes ;
- ✓ use some layout models, such as flexbox and grid, to dynamically adapt the layout ;
- ✓ make your code listen and react to events ;
- ✓ add hovers effects, image effects, transitions and animations ;
- ✓ allow users to search and/or filter tasks (by status, date, priority, etc.).

v 4.0.0